B14 Experiment tracking with MLflow

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track B: Python for data science 2
B14 Experiment tracking with MLflow 3
B14-L1 Track every run with MLflow 3
B14-L2 Registry versions and aliases, not stages 7
B14-L3 Reproduce a run from its record 11
Track B best-practice checklist 16
Appendix: solutions and answer keys 17

Track B: Python for data science

Write Python and run analyses that are correct, reproducible and easy to review.

B14 Experiment tracking with MLflow

Roadmap: Prerequisite for all months; Months 16 to 18: MLOps and deployment. Tracking, registry and the model lifecycle.

B14-L1 Track every run with MLflow

Concept

After twenty experiments nobody remembers which settings produced the good score. **MLflow tracking** records each run: parameters, metrics, tags and files. Examples use MLflow 3.16. On Databricks the tracking server is built in. Locally, MLflow 3 stores runs in a SQLite file, `mlflow.db`, in the working directory by default.

```python
import mlflow

mlflow.set_experiment("churn")
with mlflow.start_run(run_name="lr-c1"):
    mlflow.log_params({"C": 1.0, "seed": 7, "features": "v3"})
    mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
    model = train(X_train, y_train, C=1.0, seed=7)
    mlflow.log_metric("cv_auc", cv_auc)
    mlflow.sklearn.log_model(model, name="model", input_example=X_train[:5])
```

What to record, every time:

- **Parameters**: every setting that changes the result, including the seed and the feature set version. MLflow stores them as **strings**: `1.0` comes back as `'1.0'`. A parameter cannot be changed once logged in a run. Logging `C` twice with different values raises `MlflowException`.
- **Tags**: the Git commit and the input data hash from the run manifest (lesson B6-L2). They link the run to exact code and data.
- **Metrics**: the honest estimate (lesson B11-L3) and its spread, named consistently across runs: `cv_auc_mean`, `cv_auc_std`.
- **The model**: `log_model` saves the fitted object with a `requirements.txt`, `conda.yaml` and `python_env.yaml` pinned to the versions used. For scikit-learn models MLflow 3.16 saves in the skops format.

Use the `with mlflow.start_run():` block. If training raises, the run ends with status FAILED instead of staying open and collecting the next run's metrics.

Compare runs with `mlflow.search_runs(experiment_names=["churn"])`, which returns a pandas DataFrame with `params.*`, `metrics.*` and `tags.*` columns.

mlflow does not run in this app's browser runtime. The exercises check your code's structure.

Why it matters

A model with AUC 0.83 went to production from a notebook. Three months later it needed retraining. Nobody could reproduce 0.83: the notebook had been edited since, the seed had not been recorded, and the training extract had been refreshed. The team spent two weeks rebuilding a model they had already built once. One tracked run with parameters, commit and data hash would have made the retrain a single command.

Worked example: weak code

```
1 model = LogisticRegression(C=c).fit(X_train, y_train)
1 print(c, roc_auc_score(y_val, model.predict_proba(X_val)[:, 1]))
2 pickle.dump(model, open("model_final_v2.pkl", "wb"))
```

1. **Line 2.** Results live in cell output, lost on the next run.
2. **Line 3.** A file name as version control, with no code, data or environment attached.

Worked example: strong code

```
1  import mlflow
2  import numpy as np
3  from sklearn.linear_model import LogisticRegression
4
5
6  def tracked_fit(X_train: np.ndarray, y_train: np.ndarray, c: float, seed:
     int, commit: str, data_hash: str, cv_auc: float) -> str:
7      """Fit and log one run. Returns the run ID."""
1      with mlflow.start_run(run_name=f"lr-c{c}") as run:
9          mlflow.log_params({"C": c, "seed": seed})
2          mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
11         model = LogisticRegression(C=c, random_state=seed,
     max_iter=1000).fit(X_train, y_train)
12         mlflow.log_metric("cv_auc", cv_auc)
3          mlflow.sklearn.log_model(model, name="model",
     input_example=X_train[:5])
14         return run.info.run_id
```

1. **Line 8.** The run ends cleanly even if fitting raises.
2. **Line 10.** Code and data identity, attached to the run.
3. **Line 13.** The model with pinned requirements and an input example.

One run per fit, inside a with block, with parameters, commit, data hash, metrics and the model.

Recap card

- **What to log per run** Params (incl. seed), tags (git_commit, input_sha256), metrics, the model.
- **MLflow parameters** Stored as strings and immutable within a run.
- **Why with mlflow.start_run()** The run ends with FAILED status if training raises, instead of staying open.
- **log_model in MLflow 3** mlflow.sklearn.log_model(model, name="model"), with pinned requirements saved alongside.

Practice

Exercise 1. Multiple choice (level 1)

You log `mlflow.log_param("C", 1.0)` and later read the run back. What type is the value?

- a) float
- b) str
- c) int
- d) Decimal

Why?

- r1) MLflow stores parameters as strings, so compare and convert carefully.
- r2) Python cannot store floats in a database.

Exercise 2. Predict the output (level 2)

What happens? (MLflow 3.16)

What you need to know.

`mlflow.log_param(key, value)` logs one parameter to the active run. `mlflow.log_params({...})`, from the concept, logs several at once. Both write to the same parameter store, so the concept's rules about parameters apply to each call.

```
1 import mlflow
2
3 with mlflow.start_run():
4     mlflow.log_param("C", 1.0)
5     mlflow.log_param("C", 2.0)
```

Exercise 3. Fill the gap (level 2)

Log the Git commit as a tag and the model under the name `model`.

What you need to know.

Each plural MLflow call has a singular form for one item: `log_params` and `log_param(key, value)`, `log_metrics` and `log_metric(key, value)`, `set_tags` and `set_tag(key, value)`.

```
mlflow._____(1)("git_commit", commit)
mlflow.sklearn.log_model(model, _____(2)="model")
```

Exercise 4. Write the function (level 3)

Write `tracked_fit(X_train, y_train, c, seed, commit, data_hash, cv_auc) -> str`. Inside `with mlflow.start_run(...) as run:` log params `C` and `seed`, set tags `git_commit` and `input_sha256`, fit `LogisticRegression(C=c, random_state=seed, max_iter=1000)`, log the metric `cv_auc`, log the model with `mlflow.sklearn.log_model(model, name="model")`, and return `run.info.run_id`.

mlflow does not run in the browser. The checker reads your code.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Fit a logistic regression and record the run in MLflow.
def tracked_fit(
    X_train: np.ndarray,
    y_train: np.ndarray,
    c: float,
    seed: int,
    commit: str,
    data_hash: str,
    cv_auc: float,
) -> str: ...
```

Starter code:

```
import mlflow
import numpy as np
from sklearn.linear_model import LogisticRegression


def tracked_fit(X_train, y_train, c, seed, commit, data_hash, cv_auc):
    mlflow.start_run()
    model = LogisticRegression(C=c).fit(X_train, y_train)
    print(cv_auc)
    return model
```

Exercise 5. Architecture design (level 4)

Decide how each item is recorded in MLflow.

Options: Parameter, Tag, Metric, Artifact.

1. Random seed
2. Git commit
3. Cross-validated AUC
4. Calibration plot

Sources

- MLflow 3.16: default local tracking URI is sqlite:///<cwd>/mlflow.db; params come back as strings; re-logging a param with a new value raises MlflowException; log_model accepts name=; sklearn models are saved as model.skops with requirements.txt, conda.yaml and python_env.yaml. (verified: Run with MLflow 3.16.1 in a scratch environment for this repository.)

B14-L2 Registry versions and aliases, not stages

Concept

The **model registry** turns logged models into named, numbered versions. Each `log_model(..., registered_model_name="churn_model")` or `mlflow.register_model(...)` call creates a new **version**: 1, 2, 3. A version never changes. It points at one run, one set of parameters and one model file.

Aliases are movable labels on versions. MLflow 3.16:

```
from mlflow import MlflowClient

client = MlflowClient()
client.set_registered_model_alias("churn_model", "champion", "3")
model = mlflow.pyfunc.load_model("models:/churn_model@champion")
```

The scoring job always loads `@champion`. Promotion moves the alias to a new version. Rollback moves it back. The job's code never changes, and every prediction can be traced to a version and its run.

Stages (`Staging`, `Production`, `Archived`) are the older mechanism. `transition_model_version_stage` has been deprecated since MLflow 2.9, and MLflow 3.16 warns that stages will be removed. Models in Unity Catalog on Databricks use aliases only, with three-level names such as `ml.churn.churn_model` after `mlflow.set_registry_uri("databricks-uc")`.

Promote on evidence, not by hand:

1. Train and register a candidate version.

2. Evaluate it on the same held-out data and metric as the current champion.
3. Move `champion` only if the candidate beats it by a margin agreed in advance.
4. Keep the old version reachable, for example with a `previous_champion` alias, so rollback is one call.

Record the decision: which versions were compared, the scores and who approved. On Databricks, Unity Catalog grants (lesson A4-L2) control who can move an alias.

Why it matters

A team kept production models as files named `model_prod.pkl`, overwritten on each release. A new version cut conversion forecasts by 12% overnight. Rolling back meant finding last month's file on someone's laptop. It took a day. With versions and an alias, rollback is `set_registered_model_alias("churn_model", "champion", "4")`, and the scoring job picks it up on its next run.

Worked example: weak code

```
(1) client.transition_model_version_stage("churn_model", new_version,
      "Production")
    model = mlflow.pyfunc.load_model("models:/churn_model/Production")
```

1. **Line 1.** Deprecated since MLflow 2.9, not available in Unity Catalog, and promoted without a comparison.

Worked example: strong code

```
from __future__ import annotations

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from mlflow import MlflowClient


def promote_if_better(client: MlflowClient, name: str, candidate: str,
    metric: str, min_gain: float) -> bool:
    """Move the champion alias to candidate if it beats the champion by
    min_gain."""
(1)  champ = client.get_model_version_by_alias(name, "champion")
    champ_score = client.get_run(champ.run_id).data.metrics[metric]
    cand_score = client.get_run(client.get_model_version(name,
    candidate).run_id).data.metrics[metric]
(2)  if cand_score < champ_score + min_gain:
```

```
15          return False
3       client.set_registered_model_alias(name, "previous_champion",
        champ.version)
17      client.set_registered_model_alias(name, "champion", candidate)
18      return True
```

1. **Line 11.** The current champion, resolved through its alias.
2. **Line 14.** An agreed margin, so noise alone does not trigger a release.
3. **Line 16.** Rollback stays one alias move away.

Versions are fixed. Aliases move. Promote by comparison on the same data and metric, and keep the previous champion addressable.

Recap card

- **Version vs alias** A version is fixed. An alias is a movable label on a version.
- **Load the current production model** mlflow.pyfunc.load_model("models:/name@champion").
- **Model stages** Deprecated since MLflow 2.9, not supported in Unity Catalog. Use aliases.
- **Safe promotion** Compare on the same data and metric, require a margin, keep previous_champion for rollback.

Practice

Exercise 1. Multiple choice (level 1)

What does the scoring job load in an alias-based setup?

- a) `models:/churn_model/7`
- b) `models:/churn_model@champion`
- c) `models:/churn_model/Production`
- d) The latest file in a shared folder

Why?

- r1) The alias moves on promotion and rollback, so the job's code never changes.
- r2) Aliases load faster than versions.

Exercise 2. Predict the output (level 2)

A stub registry in plain Python behaves like MLflow's aliases. What does this print?

```
1 aliases = {"champion": "3"}
2 aliases["previous_champion"] = aliases["champion"]
```

```
aliases["champion"] = "5"
aliases["champion"] = aliases["previous_champion"]
print(aliases)
```

Exercise 3. Fill the gap (level 2)

Point the champion alias at version 4, then load whatever it points to.

```
client._____(1)("churn_model", "champion", "4")
model = mlflow.pyfunc.load_model("models:/churn_model_____(2)champion")
```

Exercise 4. Write the function (level 3)

Write `promote_if_better(client, name, candidate, metric, min_gain) -> bool`. The model already has a `champion` alias.

- Read the champion with `client.get_model_version_by_alias(name, "champion")`, and its score from `client.get_run(champ.run_id).data.metrics[metric]`.
- Read the candidate's score through `client.get_model_version(name, candidate)`.
- If the candidate's score is at least the champion's plus `min_gain`, set `previous_champion` to the champion's version, then `champion` to `candidate`, and return `True`. Otherwise change nothing and return `False`.

The checker passes a stub client with the same methods as `MlflowClient`.

Your code must pass these checks:

- A candidate that beats the champion by min_gain becomes champion, and the old champion becomes previous_champion.
- A gain below min_gain (0.005 < 0.01) must change nothing and return False.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Move the champion alias to candidate if it beats the champion by
    min_gain.
def promote_if_better(
    client: MlflowClient,
    name: str,
    candidate: str,
    metric: str,
    min_gain: float,
) -> bool: ...
```

Starter code:

```
def promote_if_better(client, name, candidate, metric, min_gain):
    client.set_registered_model_alias(name, "champion", candidate)
    return True
```

Exercise 5. Architecture design (level 4)

Set up model releases on Databricks with Unity Catalog.

Options: ml_prod.churn.churn_model, churn_model_final.

1. Model name
2. What the scoring job loads
3. Who can move the champion alias

Sources

- set_registered_model_alias, get_model_version_by_alias and models:/name@alias work as described; transition_model_version_stage warns it is deprecated since 2.9.0. (verified: Run with MLflow 3.16.1 in a scratch environment for this repository.)
- Models in Unity Catalog use three-level names with mlflow.set_registry_uri("databricks-uc") and do not support stages. (search-only: Databricks documentation, Manage model lifecycle in Unity Catalog (docs site not reachable from this environment).)

B14-L3 Reproduce a run from its record

Concept

A tracked run is only useful if you can rebuild its result from the record. Test that before you need it, not during an incident.

From a run ID, MLflow 3.16 gives you everything the run logged:

```
run = mlflow.get_run(run_id)
run.data.params            # {'C': '1.0', 'seed': '7', 'features': 'v3'}  strings
run.data.tags["git_commit"], run.data.tags["input_sha256"]
run.data.metrics["cv_auc"]
reqs = mlflow.pyfunc.get_model_dependencies(f"runs:/{run_id}/model")   # path to requirements.txt
```

The procedure:

1. **Code.** Check out the logged commit: `git switch --detach <commit>`.
2. **Environment.** Create a fresh environment from the lock file at that commit (lesson B6-L3), or from the model's `requirements.txt`.
3. **Data.** Load the input and compare its SHA-256 with `input_sha256`. If it differs, stop: you are reproducing on different data.

4. **Settings.** Convert the parameters back to their types. `'1.0'` must become `1.0` and `'7'` must become `7`. Passing `'7'` as a seed fails, or worse, works differently.
5. **Rerun and compare.** Recompute the metric and compare with the logged value within a small tolerance. Identical code, data, settings and versions on the same hardware usually reproduce exactly. Different CPUs or thread counts can change the last digits of floating-point results.

If the numbers differ by more than the tolerance, the record is incomplete. Something that changes the result was not logged: a feature list read from a file, a default that changed between library versions, an unseeded shuffle. Add it to the run's parameters and try again.

Schedule a reproduction check: once a quarter, pick the champion model's run and rebuild it from scratch in CI.

Why it matters

An auditor asked a lender to show how a credit model's reported AUC of 0.78 had been produced. The run existed in MLflow with parameters and a metric. The input table had since been overwritten, and no data hash had been logged. The team could not show the model had been validated on the data they claimed. The model was suspended for four weeks while it was rebuilt with full records.

Worked example: weak code

```
params = mlflow.get_run(run_id).data.params
model = LogisticRegression(C=params["C"], random_state=params["seed"])
```

1. **Line 2.** Parameters are strings. '1.0' and '7' are passed through unconverted.

Worked example: strong code

```
PARAM_TYPES: dict[str, type] = {"C": float, "seed": int, "features": str}


def parse_params(params: dict[str, str], types: dict[str, type]) ->
    dict[str, object]:
    """Convert logged string parameters back to their types. Fails on
    missing or bad values."""
    missing = sorted(set(types) - set(params))
    if missing:
        raise KeyError(f"run is missing parameters: {missing}")
    return {key: cast(params[key]) for key, cast in types.items()}


def reproduced(logged: float, rerun: float, tol: float = 1e-9) -> bool:
```

```
13      """True when the rerun metric matches the logged one within tol."""
(3)     return abs(logged - rerun) <= tol
```

1. **Line 1.** The expected parameters and their types, in one place.
2. **Line 8.** A run without a needed parameter cannot be reproduced. Say so.
3. **Line 14.** An explicit tolerance, recorded with the check.

Code from the commit, environment from the lock, data checked by hash, settings converted back to types, metric compared within a tolerance.

Recap card

- **Reproduce a run** Commit, environment from lock, data verified by hash, typed params, rerun, compare within tolerance.
- **Parameters from mlflow.get_run** Strings. Convert with the expected types before use.
- **Model dependencies** mlflow.pyfunc.get_model_dependencies(model_uri) returns the path to requirements.txt.
- **Rerun differs beyond tolerance** Something that changes results was not logged. Find it and log it.

Practice

Exercise 1. Multiple choice (level 1)

You are reproducing a run. The input file's SHA-256 differs from the logged `input_sha256`. What next?

- a) Continue. The file name is the same.
- b) Stop. You would be reproducing on different data. Find the original input.
- c) Update the logged tag to the new hash.
- d) Rerun twice and average.

Why?

- r1) Any changed byte changes the hash, so the data is not the data the run used.
- r2) Hashes are unreliable.

Exercise 2. Predict the output (level 2)

Logged parameters come back as strings. What does this print?

```
1 params = {"C": "1.0", "seed": "7"}
2 print(params["seed"] + "1", int(params["seed"]) + 1)
```

Exercise 3. Put in order (level 2)

Order the steps to reproduce a logged run.

- Load the input and verify its SHA-256 against input_sha256
- Check out the logged git commit
- Convert the logged parameters back to their types
- Build a fresh environment from the lock file at that commit
- Rerun and compare the metric within a tolerance

Exercise 4. Write the function (level 3)

Write `parse_params(params: dict[str, str], types: dict[str, type]) -> dict[str, object]`. For each key in `types`, convert `params[key]` with the type, for example `float("1.0")`. Raise `KeyError` naming the missing keys (sorted) when any key in `types` is absent from `params`. Ignore extra keys in `params`. Let a bad value raise the type's own `ValueError`.

Your code must pass these checks:

- Convert each logged string with its type.
- Missing parameters must raise KeyError naming all of them.
- Ignore extra keys, and let a bad value raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Convert logged string parameters back to their types.
def parse_params(
    params: dict[str, str],
    types: dict[str, type],
) -> dict[str, object]: ...
```

Starter code:

```python
def parse_params(params, types):
    return dict(params)
```

Exercise 5. Architecture design (level 4)

A quarterly check rebuilds the champion model's run in CI. The rerun AUC is 0.7812 against a logged 0.7851. Choose the response.

Options: Not reproduced: the gap is far above the tolerance of 1e-6, Close enough.

1. Verdict
2. Next step
3. Lasting fix

Sources

- mlflow.pyfunc.get_model_dependencies returns a path to requirements.txt; get_run(...).data.params values are strings. (verified: Run with MLflow 3.16.1 in a scratch environment for this repository.)

Track B best-practice checklist

☐ The project has a pyproject.toml, a virtual environment and pinned dependencies.

☐ ruff and mypy pass before every commit.

☐ Functions are pure where possible. Side effects sit at the edges.

☐ Dtypes are explicit. pandas code is vectorised, with no row-wise apply on large frames.

☐ Inputs are validated with pandera or pydantic at every boundary.

☐ Seeds, config and data versions are recorded for every result.

☐ Weighted estimates use the survey weights. Intervals and effect sizes go with every estimate.

☐ Preprocessing sits inside a scikit-learn Pipeline, fitted on training folds only.

☐ The metric matches the decision. Calibration and subgroup performance are checked.

☐ Every experiment is tracked in MLflow.

☐ Tests cover edge cases, including empty inputs and zeros.

☐ Tests use fixtures and parametrize, compare numbers with stated tolerances, isolate I/O, and run with coverage in CI.

☐ Every model has a model card. Every decision has a log entry.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

B14-L1 Track every run with MLflow

B14-L1 exercise 1

Hints. (1) Params and metrics are stored differently. (2) Which one is numeric? (3) Metrics.

Answer. b) str Because: r1) MLflow stores parameters as strings, so compare and convert carefully.

Parameters are strings. `run.data.params["C"]` is `'1.0'`. Convert before comparing numerically.

Common wrong answers.

- `a`: Metrics are floats. Parameters come back as strings: '1.0'.
- `c`: 1.0 is a float in Python, and MLflow stores it as the string '1.0'.
- `d`: MLflow does not use Decimal for parameters.
- `b|r2`: Right answer, wrong reason. Databases store floats. MLflow chooses strings for parameters.

B14-L1 exercise 2

Hints. (1) Can a run's parameter change after logging? (2) What would that do to reproducibility? (3) MLflow refuses.

Answer.

```
1 MlflowException
```

"Changing param values is not allowed." Start a new run for new settings.

Common wrong answers.

- `C is 2.0`: Parameters are immutable within a run. Changing a logged value raises MlflowException.
- `C is 1.0`: The second call does not pass silently. It raises.

B14-L1 exercise 3

Hints. (1) The function for one tag. (2) MLflow 3 names logged models with a keyword. (3) `set_tag` and `name`.

Answer.

```
mlflow.set_tag("git_commit", commit)
mlflow.sklearn.log_model(model, name="model")
```

Tags hold run metadata such as commits. In MLflow 3, `log_model` takes `name=` for the model's name in the run.

Common wrong answers.

- g1 = `log_param`: A commit is metadata about the run, not a setting of the model. Use a tag.

B14-L1 exercise 4

Hints. (1) Open the run with a with block. (2) log_params for settings, set_tags for commit and data hash, log_metric for the score. (3) log_model with name="model", then return run.info.run_id.

Answer.

```
import mlflow
import numpy as np
from sklearn.linear_model import LogisticRegression


def tracked_fit(X_train: np.ndarray, y_train: np.ndarray, c: float, seed: int, commit: str, data_hash: str, cv_auc: float) -> str:
    """Fit a logistic regression and record the run in MLflow.

    Returns:
        The MLflow run ID.
    """
    with mlflow.start_run(run_name=f"lr-c{c}") as run:
        mlflow.log_params({"C": c, "seed": seed})
        mlflow.set_tags({"git_commit": commit, "input_sha256": data_hash})
        model = LogisticRegression(C=c, random_state=seed, max_iter=1000).fit(X_train, y_train)
        mlflow.log_metric("cv_auc", cv_auc)
        mlflow.sklearn.log_model(model, name="model")
        return run.info.run_id
```

A run that records settings, code, data, score and model can be found, compared and reproduced.

B14-L1 exercise 5

Hints. (1) Does it change the result? (2) Is it a number to compare? (3) Is it a file?

Answer.

- Seed: a parameter, because it changes the result.
- Commit: a tag, metadata that links the run to code.

- AUC: a metric, numeric and comparable across runs.
- Plot: an artifact file.

Parameters for settings, tags for identity, metrics for numbers, artifacts for files.

Common wrong answers.

- seed = `tag`: The seed is a setting that changes the output. Log it as a parameter so searches can filter on it.
- auc = `param`: Parameters are strings. Metrics are numeric and sortable.
- commit = `metric`: A commit hash is not a number to compare. Tag it.

B14-L2 Registry versions and aliases, not stages

B14-L2 exercise 1

Hints. (1) Which reference changes without editing the job? (2) What replaced stages? (3) Aliases.

Answer. b) `models:/churn_model@champion` Because: r1) The alias moves on promotion and rollback, so the job's code never changes.

`@champion` resolves to whichever version currently holds the alias.

Common wrong answers.

- `a`: A fixed version number means editing the job for every release.
- `c`: Stages are deprecated since MLflow 2.9 and not supported in Unity Catalog.
- `d`: Files have no version history, run link or access control.
- `b|r2`: Right answer, wrong reason. Speed is the same. Indirection is the benefit.

B14-L2 exercise 2

Hints. (1) Follow each assignment in order. (2) The last line is a rollback. (3) Both point at 3.

Answer.

```
1 {'champion': '3', 'previous_champion': '3'}
```

Promotion saved the old champion, and rollback restored it in one assignment. Versions 3 and 5 are both unchanged.

Common wrong answers.

- `{'champion': '5', 'previous_champion': '3'}`: The last line rolls back: champion takes previous_champion's version, 3.

B14-L2 exercise 3

Hints. (1) The MlflowClient method that sets an alias. (2) The URI separator for aliases. (3) `set_registered_model_alias` and `@`.

Answer.

```
client.set_registered_model_alias("churn_model", "champion", "4")
model = mlflow.pyfunc.load_model("models:/churn_model@champion")
```

`set_registered_model_alias(name, alias, version)` moves the label. `models:/name@alias` loads through it.

Common wrong answers.

- g2 = `/`: models:/name/X reads X as a version or a stage. Aliases use @.

B14-L2 exercise 4

Hints. (1) Resolve the champion through its alias, then read both scores from their runs. (2) Return False early when cand_score < champ_score + min_gain. (3) Set previous_champion before moving champion.

Answer.

```
from __future__ import annotations

from typing import TYPE_CHECKING

if TYPE_CHECKING:
    from mlflow import MlflowClient


def promote_if_better(client: MlflowClient, name: str, candidate: str, metric: str, min_gain: float) -> bool:
    """Move the champion alias to candidate if it beats the champion by min_gain.

    Args:
        client: MLflow client, or any object with the same methods.
        name: Registered model name.
        candidate: Candidate version number, as a string.
        metric: Run metric to compare, higher is better.
        min_gain: Smallest improvement that justifies promotion.

    Returns:
        True if the alias moved.
    """
    champ = client.get_model_version_by_alias(name, "champion")
```

```
    champ_score = client.get_run(champ.run_id).data.metrics[metric]
    cand_score = client.get_run(client.get_model_version(name,
candidate).run_id).data.metrics[metric]
    if cand_score < champ_score + min_gain:
        return False
    client.set_registered_model_alias(name, "previous_champion",
champ.version)
    client.set_registered_model_alias(name, "champion", candidate)
    return True
```

Promotion is a comparison plus two alias moves. The margin guards against noise, and the saved alias makes rollback trivial.

B14-L2 exercise 5

Hints. (1) How are Unity Catalog objects named? (2) What replaced stages? (3) Grant to groups (lesson A4-L2).

Answer.

- Three-level names place the model in a catalog and schema, governed like tables.
- Jobs load by alias. Unity Catalog models do not support stages.
- Grant the right to move aliases to a release group, not to every trainer.

Govern models like tables: three-level names, aliases for releases, group grants for who can release.

Common wrong answers.

- pointer = `stage`: Stages are deprecated and not supported for models in Unity Catalog.
- who = `anyone`: Moving the alias changes production. Limit it to a group, as with any production change.
- name = `plain`: Unity Catalog models use catalog.schema.model names.

B14-L3 Reproduce a run from its record

B14-L3 exercise 1

Hints. (1) What does a different hash mean? (2) Can you reproduce on different data? (3) Find the original.

Answer. b) Stop. You would be reproducing on different data. Find the original input. Because: r1) Any changed byte changes the hash, so the data is not the data the run used.

A hash mismatch means different data. Reproduction needs the original input, which is why raw inputs are kept immutable (lesson B6-L2).

Common wrong answers.

- `a`: Same name, different content. The hash exists for exactly this check.
- `c`: Rewriting the record destroys the evidence of what the run used.
- `d`: Averaging runs on the wrong data does not recover the right result.
- `b|r2`: Right answer, wrong reason. The hash is reliable. That is why the mismatch matters.

B14-L3 exercise 2

Hints. (1) What does + do with two strings? (2) And with two ints? (3) Concatenation, then addition.

Answer.

```
1 71 8
```

Unconverted parameters behave like text. Convert them back to their types before use.

Common wrong answers.

- `8 8`: '7' + '1' joins strings: '71'. Convert with int() first.

B14-L3 exercise 3

Hints. (1) Which step provides the lock file? (2) Which check should stop you early? (3) Compare last.

Answer.

1. Check out the logged git commit
2. Build a fresh environment from the lock file at that commit
3. Load the input and verify its SHA-256 against input_sha256
4. Convert the logged parameters back to their types
5. Rerun and compare the metric within a tolerance

Code, then environment, then data, then settings, then the comparison.

Common wrong answers.

- The lock file that defines the environment lives at the commit. Check it out first.
- Verify the data before rerunning, or a mismatch wastes the run and confuses the comparison.

B14-L3 exercise 4

Hints. (1) Find missing keys with a set difference. (2) Raise KeyError with the sorted list. (3) `{key: cast(params[key]) for key, cast in types.items()}`.

Answer.

```python
def parse_params(params: dict[str, str], types: dict[str, type]) ->
    dict[str, object]:
    """Convert logged string parameters back to their types.

    Args:
        params: Parameters as MLflow returns them.
        types: Expected keys and the type to convert each to.

    Returns:
        Typed parameters for the expected keys.

    Raises:
        KeyError: If an expected key is missing.
        ValueError: If a value cannot be converted.
    """
    missing = sorted(set(types) - set(params))
    if missing:
        raise KeyError(f"run is missing parameters: {missing}")
    return {key: cast(params[key]) for key, cast in types.items()}
```

Explicit types turn the run record back into the exact settings. Failing on a missing key beats silently rerunning with a default.

B14-L3 exercise 5

Hints. (1) Is 0.004 rounding? (2) What does the record not contain? (3) Fix the record, not the check.

Answer.

- A gap of 0.004 is a real difference, not rounding.
- Compare every recorded identity, then look for settings that were never logged.
- Log what was missing, so the next check passes for the right reason.

A failed reproduction points at a gap in the record. Close the gap, keep the check.

Common wrong answers.

- verdict = `fine`: 0.004 AUC is larger than many model improvements you would ship. Something differs.
- next = `relog`: Editing the old run erases the evidence of what it produced.
- fix = `widen`: A wide tolerance hides incomplete records instead of fixing them.